

P2384R1

2021 Spring Library Evolution Poll

Outcomes

Published Proposal, 2021-09-25

Author:

[Bryce Adelstein Leibach \(he/him/his\)](#) — [Library Evolution Chair](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

- 1 Introduction**
- 2 Poll Outcomes**
- 3 Selected Poll Comments**
 - 3.1 Poll 1: [P0323R10] expected
 - 3.2 Poll 2: [P2325R2] Views Should Not Be Required To Be Default Constructible
 - 3.3 Poll 3: [P2328R0] `ranges::join_view` Should Join All Views Of Ranges
 - 3.4 Poll 4: [P2210R2] Superior String Splitting
 - 3.5 Poll 5: [P2321R1] `views::zip`
 - 3.6 Poll 6: [P2251R1] Require `span & basic_string_view` To Be Trivially Copyable
 - 3.7 Poll 7: [P1072R7] `basic_string::resize_and_overwrite`

3.8 Poll 8: [P2340R0] Clarifying The Status Of The "C Headers"

3.9 Poll 9: [P2301R0] Add A pmr Alias For stacktrace

References

Informative References

§ 1. Introduction

In Spring 2021, the C++ Library Evolution group conducted a series of electronic decision polls [P2368R1]. This paper provides the results of those polls and summarizes the results.

In total, 33 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send [P0323R10] (expected) to Library Working Group for C++23 instead of the proposed C++ Library Fundamentals Technical Specification version 3, classified as an addition ([P0592R4] bucket 3 item).	11	11	1	2	0	Consensus in favor.
Poll 2: Send [P2325R2] (Views Should Not Be Required To Be Default Constructible) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item), with the recommendation that implementations retroactively apply it to C++20.	21	7	0	0	0	Unanimous consensus in favor.

Poll 3: Send [P2328R0] (ranges::join_view Should Join All Views Of Ranges) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item), with the recommendation that implementations retroactively apply it to C++20.	17	8	0	0	0	Unanimous consensus in favor.
Poll 4: Send [P2210R2] (Superior String Splitting) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item), with the recommendation that implementations retroactively apply it to C++20.	21	8	0	0	0	Unanimous consensus in favor.
Poll 5: Send [P2321R1] (views::zip) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	19	7	0	1	0	Consensus in favor.
Poll 6: Send [P2251R1] (Require span & basic_string_view To Be Trivially Copyable) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	21	10	0	0	0	Unanimous consensus in favor.
Poll 7: Modify [P1072R7] (basic_string::resize_and_overwrite) by adding a feature test macro, and then send the revised paper to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	17	8	0	1	1	Consensus in favor.
Poll 8: Send [P2340R0] (Clarifying The Status Of The "C Headers") to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	13	8	2	0	1	Consensus in favor.
Poll 9: Send [P2301R0] (Add A pmr Alias For stacktrace) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	6	14	5	0	0	Strong consensus in favor.

§ 3. Selected Poll Comments

§ 3.1. Poll 1: `[P0323R10]` `expected`

This paper has been under revision by Library Evolution since 2014 and follows the well-established design of `optional`, as Library Evolution asked the paper authors to do. The paper shows substantial usage with multiple implementations, and the feedback received tells us that the feature is ready for production use. A TS will not provide information which we do not already have.

— Strongly Favor

I do not believe that TSeS with a broad and non-specific focus like the Library Fundamentals TS serve us well. It's not clear what we would learn from the TS. There is a great desire for something like `expected`, as evidenced by the many different forms of it that exist in the wild today. While there are alternative design decisions, and other approaches, like the proposed deterministic exception handling (P0709), I do not think we need to wait. There's sufficient field experience and demand for this to ship it now.

— Strongly Favor

C++ needs a good type for expressing error conditions for when exceptions are not appropriate. Text formatting (P1729) and process management (P1750) could each benefit significantly from such a type. SG14 has been wanting such a type for some time.

— Strongly Favor

I've used `tl::expected`, which is based on this paper, in multiple projects successfully. The community has expressed an inability to use exceptions in many areas (embedded, games) and efforts to improve exceptions performance, portability and determinism have yet to go anywhere. It is also very difficult to use exceptions correctly in asynchronous or parallel code. This type solve all of these issues, at least to a degree that is much more reasonable than the status quo.

— Weakly Favor

Our company has internally had requests for a type that does something along the lines of this type.

— Weakly Favor

If we don't advance this paper and it gets bogged down in further design review, we'll just never have it. There were two issues brought up in the discussion: `expected<T, E>`'s conversion to `bool` and `expected<T, E>`'s comparisons to `T`. Both are potentially problematic, though the former at least has a lot of value while the latter seems like a bug magnet. But the potential problems with either are surely dwarfed by the value of the facility as a whole... so we should at least try to ship it... ever.

— Weakly Favor

We are in dire need of standardized error-or-value type, but there is still a lot of competing types in the area, and discussion regarding the API. I am not sure if this design is mature enough to vote it in now.

— Neutral

There have been extended discussions on the design of an `expected`-like facility. I'd prefer to see a trial in a TS instead of going straight to IS.

— Weakly Against

To me this facility doesn't have common convincing use-cases to be included in C++ standard. I know that there was a long discussion and people see the value of that feature but from my perspective it is not super generic. Basing on my experience with building the error channel using this facility it was not very convenient and looks more like the code pollution forcing me to write `expected` everywhere I want error propagation for on the higher level and then, forcing the user to do the same where the error from my library should be propagated on higher level of the user code. It's debatable if explicit (visible) error channel is better than the hidden one or not. I don't want to dive into that holy war now.

Although, I understand that with `expected` we have a combination of the exact return value and the error and even allow specifying that error type, to me it's still looks closer to C approach when we get an error from the function return value. Also I am concerned that after each call of the function with `expected` return type I should write the if-check if I want to deal with error. That doesn't sound like ""Pay as you go"" principle. Exceptions for example (zero overhead implementation) don't bring this requirement and don't add the overhead unless they are thrown (they do affect the binary size, but in general I don't care much). I may test my code in some situations and make sure exception is never thrown and then don't write try-catch at all.

What I am also afraid of is adding this facility to C++ standard enables one more orthogonal way to add error channel that should somehow interoperate with the existing code, which might end up with inconvenient conversions from one error mechanism to another (e.g. from exceptions to `expected` or vice versa) on different application layers because some libraries start using new error channel, others continue using exceptions, somebody else invents another "very fancy" way to notify about the error. And what's more important Exceptions remain the primary error channel for C++ standard library that also might (and eventually, will) add some problems with interoperability and the mess for the user code. I believe we should move toward unification of the error channel that works well for all (or almost all) scenarios rather than introducing one more way for not very large group of use-cases but definitely increasing the disunity.

I am (almost 100%) sure that my vote doesn't change anything but I want to express my option honestly.

— Weakly Against

§ 3.2. Poll 2: [P2325R2] Views Should Not Be Required To Be Default Constructible

This fixes a known flaw in ranges in C++20. Luckily, because no implementation has shipped ranges and the concepts that support them yet, we have a chance to fix this, and we should take it.

— Strongly Favor

Requiring Regular turns out to be an overly restrictive requirement, especially in the face of spans of fixed size which would otherwise be perfectly good views.

— Strongly Favor

Adding a default constructor to a class that isn't naturally default constructible is, in my opinion, a code smell. The benefit of allowing non-default constructible things such as `span` to be views outweighs the cost of changing a concept.

— Strongly Favor

This is a must have because it finally makes all `span` views and there is really no reason to limit the construction of a type in any form.

— Strongly Favor

§ 3.3. Poll 3: [P2328R0] `ranges::join_view` Should Join All Views Of Ranges

This is a better design for `ranges::join_view`, and we are still in the window when we can likely apply it to C++20.

— Strongly Favor

It's good to fix this, especially if we apply those retroactively to existing implementations.

— Strongly Favor

I'm convinced this is important. I hope we're making the right fix and that our experience with `ranges::` is enough.

— Weakly Favor

I encourage the general direction of loosely interpreting the $O(1)$ requirements for range operations, but am not expert in this particular example.

— Chose to Not Vote

§ 3.4. Poll 4: [P2210R2] Superior String Splitting

The motivation and rationale for this change is convincing. Here in particular, we gain a split that actually works in practice in an easy-to-use manner, which is something we've been waiting for for decades. The original design rationale was a fine idea, but we need to be able to make such design compromises in order to ship a useful split facility.

— Strongly Favor

`split_view` as currently specified is borderline unusable for the most common use case of splitting strings; this rectifies the problem.

— Strongly Favor

That strings couldn't be sensibly split was a defect. Fixing the defect is a good thing.

— Weakly Favor

I support the paper, but `lazy_split` isn't a good name. I've reached out to the paper author to bikeshed some names.

— Strongly Favor

§ 3.5. Poll 5: [P2321R1] `views::zip`

Probably the most requested view, and it's impossible to implement in a user library without the changes to `tuple` that this paper makes.

— Strongly Favor

I support the direction to add `views::zip` to the standard library, because it's the most frequent ranges enhancement request I receive from peers.

— Weakly Favor

I find this feature useful in other programming languages, and would very much like it in C++ ranges as well.

— Strongly Favor

This feature is among the most important missing pieces of ranges functionality. The complexity of the proxy-iterator interface is unfortunate, but that is not enough to recommend against it.

— Weakly Favor

Not great to further complicate tuple & pair constructors.

— Weakly Favor

This paper has open design questions on Library Evolution reflector, that should be clarified before going to Library. I am not opposed by to having the paper.

— Weakly Against

§ 3.6. Poll 6: [P2251R1] Require `span` & `basic_string_view` To Be Trivially Copyable

These types are deliberately trivial (in the English sense); implementations have nothing to gain from making them non-trivial (in the C++ sense), and programmers will derive substantive benefits from the guarantee.

— Strongly Favor

This is both correct and useful (in particular for computation on GPUs, where one has to `memcpy` spans-of-device-allocations into GPU memory).

— Strongly Favor

This is a good thing if it doesn't have ABI breaking consequences. I believe that author did the analysis about ABI stability, so I don't see the showstoppers for it to be shipped.

— Weakly Favor

My only concern is that there may be other types where we strongly imply that a type is trivially copyable, which was the style of wording to require that two decades ago, but it seems we are being more specific in new types.

— Weakly Favor

§ 3.7. Poll 7: [P1072R7] `basic_string::resize_and_overwrite`

The functionality is useful for performance. The name is consistent with `make_unique_for_overwrite`. I wish I had this for `vector`, and I'm hopeful that this will lead to a pattern for containers like `vector`.

— Strongly Favor

This is the best interface given that we want to minimize the window where the string has uninitialized data (which state adds new preconditions on various string operations).

— Strongly Favor

This is very sharp-knife made for very specific purpose. I think the interface proposed reflects that property.

— Strongly Favor

It plugs a performance hole that otherwise drives our users to re-invent strings or to invoke Undefined Behavior for optimization's sake.

— Strongly Favor

This will help with a lot of C interoperability use cases. Adding a feature test macro seems natural and like something users would expect.

— Weakly Favor

In a technical sense, this feature weakens the type system in that it introduces a new, invalid state for `string`. However, there are already such states (e.g., strings that are out of lifetime or that are subject to concurrent access), and the indirect nature of the interface serves to reinforce the special nature of the intermediate state and discourage misuse. That indirection is merely acceptable in terms of teachability, but the feature is plainly important for performance of real applications.

— Weakly Favor

The proposed solution is more complex (for the end user more than for the implementation) than is warranted for the problem that is being solved. I would rather see the problem remain unsolved than adopt this solution.

— Weakly Against

Undefined behavior if a callback throws is unprecedented. The prior art does not use this design.

— Strongly Against

§ 3.8. Poll 8: [P2340R0] Clarifying The Status Of The "C Headers"

This instance of deprecation is plainly causing more harm (in terms of misunderstanding) than good. The direction chosen, while unavoidably imprecise, is surely the best option available.

— Strongly Favor

The line between "style guide" and "descriptive, not prescriptive standard" is fine. While I fully support this proposal for the C headers, I don't want to see it expanded or generalized into a coding standard, because that isn't the business the C++ Committee should be in.

— Strongly Favor

We must face reality: the C headers are not going away...

— Strongly Favor

Allowing users to choose to write valid ISO C which is also valid ISO C++ seems useful for maintaining the tight connection/interoperation the two languages enjoy.

— Weakly Favor

I'm ambivalent. We keep flip-flopping on whether `stdint.h` vs. `cstdint` is good practice, and I do not know how to feel about it.

— Neutral

C headers were an unfortunate mistake from the beginning. It is as much a technical issue (these interfaces are terrible, yet some names are very good), as it is an organizational one (the C++ committee doesn't control these headers).

Un-deprecating them feels like giving up and sends the wrong message. It will do nothing to dissuade people to use them, quite the contrary, and it does not give us back the control to modify `std::strlen` without an unreasonable amount of difficulty for a change that would otherwise take a few minutes.

The guarantees we provide are hardly guarantees we can make, as the C committee could decide tomorrow to change these interfaces without consulting us. As long as there are 2 different bodies taking different votes and stances on language design, we should try to find a way so that C++ can evolve in the direction it wants (e.g. making things `constexpr` and `noexcept` where it makes sense notably).

We should instead entertain documenting in the C++ standard the `<xxxx>` headers, so that we have control over the wording, remove the requirements that these functions have identical addresses as their C counterparts (we usually don't support people taking the address of `std::` functions).

This paper does not solve any problem.

— Strongly Against

§ 3.9. Poll 9: [P2301R0] Add A `pmr` Alias For `stacktrace`

To my understanding, such an alias is useful to a group of people and cannot be created externally to the language. But I would love to see the motivation in R1.

— Weakly Favor

It takes longer to write this comment than to review the wording in the paper. It's a consistency improvement.

— Weakly Favor

Unconvinced about the importance of this.

— Weakly Favor

Meh.

— Neutral

This is a small thing that is super easy to implement. I don't have objections, but I am also is not supportive.

— Neutral

While this is "just a convenience", I have not weighed the complexity of any such extension against the consistency arguments with other `pmr::` containers.

— Chose to Not Vote

§ References

§ Informative References

[P0323R10]

JF Bastien, Vicente Botet. [*std::expected*](https://wg21.link/p0323r10). 15 April 2021. URL: <https://wg21.link/p0323r10>

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P1072R7]

Chris Kennelly, Mark Zeren. *basic_string::resize_and_overwrite*. 15 February 2021. URL: <https://wg21.link/p1072r7>

[P2210R2]

Barry Revzin. *Superior String Splitting*. 5 March 2021. URL: <https://wg21.link/p2210r2>

[P2251R1]

Nevin Liber. *Require span & basic_string_view to be Trivially Copyable*. 19 March 2021. URL: <https://wg21.link/p2251r1>

[P2301R0]

Steve Downey. *Add a pmr alias for std::stacktrace*. 15 February 2021. URL: <https://wg21.link/p2301r0>

[P2321R1]

Tim Song. *zip*. 11 April 2021. URL: <https://wg21.link/p2321r1>

[P2325R2]

Barry Revzin. *Views should not be required to be default constructible*. 23 April 2021. URL: <https://wg21.link/p2325r2>

[P2328R0]

Tim Song. *join_view should join all views of ranges*. 15 March 2021. URL: <https://wg21.link/p2328r0>

[P2340R0]

Thomas Köppe. *Clarifying the status of the ‘C headers’*. 15 March 2021. URL: <https://wg21.link/p2340r0>

[P2368R1]

Bryce Adelstein Leibach. *2021 Spring Library Evolution Polls*. 28 May 2021. URL: <https://wg21.link/p2368r1>